SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

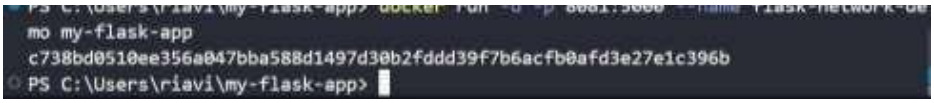
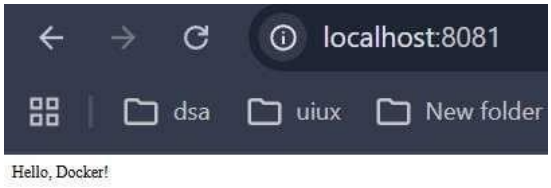
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 09

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Pushkraj P Naik**
PRN No.	**23070122169**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025 – 2026, Semester VII
Date of Performance	23rd July, 2026
Title of Assignment:	Container Management & Networking

Theory:	Docker container management is the process of creating, running, monitoring, stopping, inspecting, and removing containers. Port mapping allows applications running inside a container to be accessed from the host machine by mapping a host port to a container port. Docker provides commands such as docker ps to view running containers, docker inspect to examine detailed container configuration and network settings, docker stop to gracefully stop a container, and docker rm to remove unused containers. These commands help efficiently manage the lifecycle of Docker containers and verify their network configuration and status. Commands Used • Created and started the Docker container in detached mode with port mapping: docker run -d -p 8081:5000 --name flask-network-demo my-flask-app • Listed all running Docker containers: docker ps • Stopped the running Docker container: docker stop flask-network-demo • Listed all Docker containers, including stopped containers: docker ps -a • Inspected the container's network configuration and settings: docker inspect flask-network-demo • Removed the stopped Docker container: docker rm flask-network-demo

	· Verified that the container had been removed: docker ps -a

Implementation:	**Step 1: Run a Container with Port Mapping**  Started Docker Container — Started the Docker container named flask-networkdemo in detached mode and mapped host port 8081 to container port 5000, allowing the application to run in the background. **Step 2: Access the Application via Your Browser**  Verified Application Access — Accessed the Dockerized Flask application through http://localhost:8081 and verified that the application was successfully running inside the Docker container. **Step 3: List Running Containers**  Listed Running Containers — Displayed all currently running Docker containers using the docker ps command and verified that the flask-network-demo container was running with port mapping from 8081 on the host to 5000 inside the container. **Step 4: Stop the Running Container**  Stopped Docker Container — Stopped the running flask-network-demo container using the docker stop command, terminating the application gracefully. **Step 5: List Running and All Containers**

```
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % docker network cr
8afb6a7b85a950cd53383d4bc3acf7ccc80f642b27e50c86a17a4e5e981bfa42
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % docker run -d --name web-container --network devops-network nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
d3531b7a4f21: Pull complete
cbe54b031f3e: Pull complete
1408044c7403: Pull complete
7b9d02243238: Pull complete
f523b340a6c0: Pull complete
dbb1ef19121a: Pull complete
567a969e0221: Pull complete
c2fefbef5314: Pull complete
2d759c5f2f47: Download complete
db5a6d6bb46f: Download complete
Digest: sha256:a3554c0d0b40177b1413c26e01f695334428020d9df01243fbae3e11cfd17417
Status: Downloaded newer image for nginx:alpine
f12983d64f82516caaa73374b91db25e8c45cd6c305deec23307d7a322aec26f
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % docker run -d --name test-container --network devops-network python:3.12-sl
leep infinity
Unable to find image 'python:3.12-slim' locally
3.12-slim: Pulling from library/python
Digest: sha256:a0f57064cbdcd6285fa45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704
Status: Downloaded newer image for python:3.12-slim
15485eb0d96a6d6ca730f16c2a117ce4f5874ca48d02ff6550de38a7dd32dfe9
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % docker exec test-container python -c "import urllib.request; print(urllib.r
st.urlopen('http://web-container', timeout=5).status)"
200
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % docker ps --filter "network=devops-network"
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS          PORTS     NAMES
15485eb0d96a   python:3.12-slim  "sleep infinity"         11 seconds ago   Up 9 seconds              test-container
f12983d64f82   nginx:alpine      "/docker-entrypoint.…"   19 seconds ago   Up 17 seconds   80/tcp    web-container
(base) pushkrajpnaik@Pushkrajs-Laptop Assignment-09-Container-Networking % git rev-parse --show-toplevel
/Users/pushkrajpnaik/Desktop/Pushkraj /DevOps
```

Verified Container Status — Used the docker ps -a command to display all Docker containers and verified that the flask-network-demo container had stopped
successfully and was in the Exited state.

Step 6: Inspect the Container's Network Settings

```
PS C:\Users\riavi\my-flask-app> docker inspect flask-network-demo
        },
        "NetworkSettings": {
            "SandboxID": "",
            "SandboxKey": "",
            "Ports": {},
            "Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "ae7b1c98448c112eee0784f77a5090772b740ff5be9fedf421799e0803a4
700",
                    "EndpointID": "",
                    "Gateway": "",
                    "IPAddress": "",
                    "MacAddress": "",
                    "IPPrefixLen": 0,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": null
                }
            }
        },
```

Inspected Container Network Settings — Used the docker inspect command to view detailed information about the flask-network-demo container, including its NetworkSettings and network configuration.

Step 7: Remove the Stopped Container

```
PS C:\Users\riavi\my-flask-app> docker rm flask-network-demo
flask-network-demo
PS C:\Users\riavi\my-flask-app> docker ps -a
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS
               PORTS
               NAMES
ac7bba30e105   my-flask-app             "python app.py"          38 minutes ago   Exited (0)
 14 minutes ago
               festive_morse
5195a83ad0e8   785e6affefd7             "python app.py"          41 minutes ago   Exited (1)
 41 minutes ago
               relaxed_williams
defd1b9f4c1a   jenkins/jenkins:lts-jdk17   "/usr/bin/tini -- /u…"   6 days ago       Up 5 hours
               0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:5
0000->50000/tcp   myjenkins
63d8b2cf90b8   hello-world              "/hello"                 6 days ago       Exited (0)
 6 days ago
               funny_dirac
PS C:\Users\riavi\my-flask-app>
```

Removed Docker Container — Removed the flask-network-demo container using the docker rm command and verified its successful removal by confirming that it no longer appeared in the list of Docker containers.

Subject Faculty Name: Prof. Arijit Dutta